

UNIT – V EXCEPTION HANDLING AND GRAPHICS

Date & Time methods, Exception Handling Clauses, Raising an Exception, User Defined Exceptions; Simple Graphics and Image Processing, Simple 2D drawings- colors and shapes.

I. Date and Time Methods

Python has a module named **datetime** to work with dates and times.

Example 1: Get Current Date and Time

Eg.
import datetime
print(datetime.datetime.now())
O/P:
2023-05-29 11:40:04.611729

Example 2: Get Current Date

Eg.
import datetime
print(datetime.date.today())
O/P:
2023-05-29

Commonly used classes in the datetime module are:

- **date class**– Manipulate just date (Month, day, year)
- **time class** – Time independent of the day (Hour, minute, second, microsecond)
- **datetime class** – Combination of time and date (Month, day, year, hour, second, microsecond)
- **timedelta class** — A duration of time used for manipulating dates

date class

We will use the `date` class of the `datetime` module to accomplish this task.

Example 1: Python get today's date

```
from datetime import date
today = date.today()
print("Today's date:", today)
```

Output:
Today's date: 2023-05-29

Here, we imported the `date` class from the `datetime` module. Then, we used the `date.today()` method to get the current local date.

By the way, `date.today()` returns a `date` object, which is assigned to the `today` variable in the above program.

Now, we can use the `strftime()` method to create a string representing date in different formats.

- `strftime()` method is used for formatting.
- This method uses different **control code** to give an output.
- Each control code resembles different parameters like year, month, weekday and date `[(%y/%Y – Year), (%a/%A- weekday), (%b/%B- month), (%d - day of month)]` .

Example 2: Current date in different formats

```
from datetime import date

today = date.today()

# dd/mm/YY
d1 = today.strftime("%A - %d/%m/%Y")
print("d1 =", d1)

# Textual month, day and year
d2 = today.strftime("%B %d, %Y")
print("d2 =", d2)

# mm/dd/y
d3 = today.strftime("%m/%d/%y")
print("d3 =", d3)

# Month abbreviation, day and year
d4 = today.strftime("%b-%d-%Y")
print("d4 =", d4)
```

O/P:

```
d1 = Monday - 29/05/2023
d2 = May 29, 2023
d3 = 05/29/23
d4 = May-29-2023
```

Today's Weekday Number

The `date.today()` function also gives you the weekday number. Here is the Weekday Table which start with Monday as 0 and Sunday as 6

Day	WeekDay Number
Monday	0
Tuesday	1
Wednesday	2
Thursday	3
Friday	4
Saturday	5
Sunday	6

Eg.

```
from datetime import date
today=date.today()
print("Todays day is:", today.day )
print("Todays month is:", today.month )
print("Todays year is:", today.year )
print("Todays weekday is:", today.weekday())
```

O/P:

```
Todays day is: 29
Todays month is: 5
Todays year is: 2023
Todays weekday is: 0
```

datetime class

If we need to get the current date and time, we can use `datetime` class of the `datetime` module.

Example 3: Get the current date and time

```
from datetime import datetime

# datetime object containing current date and time
now = datetime.now()

print("now =", now)
```

```
# dd/mm/YY H:M:S
dt_string = now.strftime("%d/%m/%Y %H:%M:%S")
print("date and time =", dt_string)
```

Output:

```
now = 2023-05-29 11:48:12.985487
date and time = 29/05/2023 11:48:12
```

✓ With the help of "Strftime" function we can also retrieve **local system time, date or both.**

1. %C- indicates the local date and time
2. %x- indicates the local date
3. %X- indicates the local time

Eg.

```
from datetime import datetime
now = datetime.now()
print(now.strftime("%c"))
print(now.strftime("%x"))
print(now.strftime("%X"))
```

O/P:

```
Mon May 29 11:48:56 2023
```

```
05/29/23
```

```
11:48:56
```

✓ The "strftime function" allows us to call the time in any **format 24 hours or 12 hours.**

Eg.

```
from datetime import datetime
now = datetime.now()
print(now.strftime("%I: %M:%S %p"))
#%I/%H 12-hour/24 hour, %M - Minute, %S -
Second, %p - local' AM/PM
print(now.strftime("%H:%M"))
```

O/P:

```
11: 49:56 AM
```

```
11:49
```

time class

With "DATETIME OBJECT", we can also call time class.

Suppose we want to print just the current time without the date.

```
t = datetime.time(datetime.now())
```

- We had imported the time class. We will be assigning it the current value of time using `datetime.now()`
- We are assigning the value of the current time to the variable `t`.

And this will give just the time.

Eg.

```
from datetime import datetime
from datetime import time
t=datetime.time(datetime.now())
print("The current time is ",t)
```

O/P:

The current time is 11:51:07.712635

How to use Timedelta Objects

With timedelta objects, we can estimate the time for both future and the past. In other words, it is a timespan to predict any special day, date or time.

Remember this function is not for printing out the time or date, but something to CALCULATE about the future or past. Let's see an example to understand it better.

Step 1) To run Timedelta Objects, we need to declare the import statement first and then execute the code

Eg.

```
from datetime import date
from datetime import time
from datetime import datetime
from datetime import timedelta
```

```
print(timedelta(days=365, hours=12, minutes=18))
```

O/P:

365 days, 12:18:00

The timedelta represents a span of 365 days, 12 hrs and 18 minutes and prints the same

Step 2) Let's get today's date and time to check whether our import statement is working well. When code is executed, it prints out today's date which means our import statement is working well

Eg.

```
from datetime import date
from datetime import time
from datetime import datetime
from datetime import timedelta

print(timedelta(days=365, hours=12, minutes=18))

print ("today is :",str(datetime.now()))
```

O/P:

```
365 days, 12:18:00
today is : 2023-05-29 11:52:17.518660
```

Step 3) We can retrieve date, a year from now through delta objects.

Eg.

```
from datetime import date
from datetime import time
from datetime import datetime
from datetime import timedelta
print(timedelta(days=365, hours=12, minutes=18))

print ("today is :",str(datetime.now()))

print("One year from now will be:",
datetime.now()+timedelta(days=365))
```

O/P:

```
365 days, 12:18:00
today is : 2023-05-29 11:52:53.983545
One year from now will be: 2024-05-28
11:52:53.983545
```

Step 4) Another example of how time delta can be used to calculate future date from current date and time

Eg.

```
from datetime import date
from datetime import time
from datetime import datetime
from datetime import timedelta
```

```
print(timedelta(days=365, hours=12, minutes=18))
```

```
print ("today is :",str(datetime.now()))
```

```
print("One year from now will be:", datetime.now()+timedelta(days=365))
```

```
print("In 1 week and 2 days it will be:", datetime.now()+timedelta(weeks=1,
days=2))
```

O/P:

```
365 days, 12:18:00
```

```
today is : 2023-05-29 11:53:30.132044
```

```
One year from now will be: 2024-05-28 11:53:30.132044
```

```
In 1 week and 2 days it will be: 2023-06-07 11:53:30.147669
```

Step 5) Let's look into a more complex example. Determine how many days past the New Year.

Eg.

```
from datetime import date
from datetime import time
from datetime import datetime
from datetime import timedelta
```

```
today=date.today()
print("today", today)
```

```
nyd=date(today.year,1,1)
print("nyd",nyd)
```

```
if nyd<today:
    print("new year is already went by %d days ago" % ((today-nyd).days))
else:
    print("%d days more for new year"%((nyd-today).days))
```

O/P:

```
today 2023-05-29
```

```
nyd 2023-01-01
```

```
new year is already went by 148 days ago
```

Getting calendar for a month

The calendar module gives a wide range of methods to play with yearly and monthly calendars. Here, we print a calendar for a given month

Eg.

```
import calendar  
  
cal = calendar.month(2020, 4)  
  
print ("Here is the calendar:")  
  
print (cal)
```

O/P:

Here is the calendar:

April 2020

Mo Tu We Th Fr Sa Su

1 2 3 4 5

6 7 8 9 10 11 12

13 14 15 16 17 18 19

20 21 22 23 24 25 26

27 28 29 30

II. ERRORS AND EXCEPTIONS

There are two distinguishable kinds of errors: *syntax errors and exceptions*.

Syntax Errors

Syntax errors, also known as parsing errors, are perhaps the most common kind of complaint you get while you are still learning Python. **Syntax error** is an error in the syntax of a sequence of characters or tokens that is intended to be written in python. For compiled languages, syntax errors are detected at compile-time. A program will not compile until all syntax errors are corrected. For interpreted languages, however, a syntax error may be detected during program execution, and an interpreter's error messages might not differentiate syntax errors from errors of other kinds.

Exceptions

Even if a statement or expression is syntactically correct, it may cause an error when an attempt is made to execute it. Errors detected during execution are called *exceptions*. You will soon learn how to handle them in Python programs. Most exceptions are not handled by programs, however,

result in error messages as shown here:

```
>>> 55+(5/0)
Traceback (most recent call last):
  File "<pyshell#12>", line 1, in
    <module> 55+(5/0)
ZeroDivisionError: integer division or modulo by zero

>>> 5+ repeat*2
Traceback (most recent call last):
  File "<pyshell#13>", line 1, in
    <module> 5+ repeat*2
NameError: name 'repeat' is not defined

>>> '5'+5
Traceback (most recent call last):
  File "<pyshell#14>", line 1, in
    <module> '5'+5
TypeError: cannot concatenate 'str' and 'int' objects
```

The last line of the error message indicates what happened. Exceptions come in different types, and the type is printed as part of the message: the types in the example are **ZeroDivisionError**, **NameError** and **TypeError**.

The string printed as the exception type is the name of the built-in exception that occurred. This is true for all built-in exceptions, but need not be true for user-defined exceptions (although it is a useful convention). Standard exception names are built-in identifiers (not reserved keywords).

The preceding part of the error message shows the context where the exception happened, in the form of a stack traceback. In general it contains a stack traceback listing source lines; however, it will not display lines read from standard input.

Exception Handling Clauses

EXCEPTION NAME	DESCRIPTION
Exception	Base class for all exceptions
StopIteration	Raised when the next() method of an iterator does not point to any object.
SystemExit	Raised by the sys.exit() function.
StandardError	Base class for all built-in exceptions except StopIteration and SystemExit.
ArithmeticError	Base class for all errors that occur for numeric calculation.
OverflowError	Raised when a calculation exceeds maximum limit for a numeric type.
FloatingPointError	Raised when a floating point calculation fails.
ZeroDivisionError	Raised when division or modulo by zero takes place for all numeric types.
AssertionError	Raised in case of failure of the Assert statement.
AttributeError	Raised in case of failure of attribute reference or assignment.
EOFError	Raised when there is no input from either the raw_input() or input() function and the end of file is reached.
ImportError	Raised when an import statement fails.
KeyboardInterrupt	Raised when the user interrupts program execution, usually by pressing Ctrl+c.

LookupError	Base class for all lookup errors.
IndexError	Raised when an index is not found in a sequence.
KeyError	Raised when the specified key is not found in the dictionary.
NameError	Raised when an identifier is not found in the local or global namespace.
UnboundLocalError	Raised when trying to access a local variable in a function or method but no value has been assigned to it.
EnvironmentError	Base class for all exceptions that occur outside the Python environment.
IOError	Raised when an input/ output operation fails, such as the print statement or the open() function when trying to open a file that does not exist.
OSError	Raised for operating system-related errors.
SyntaxError	Raised when there is an error in Python syntax.
IndentationError	Raised when indentation is not specified properly.
SystemError	Raised when the interpreter finds an internal problem, but when this error is encountered the Python interpreter does not exit.
SystemExit	Raised when Python interpreter is quit by using the sys.exit() function. If not handled in the code, causes the interpreter to exit.
TypeError	Raised when an operation or function is attempted that is invalid for the specified data type.
ValueError	Raised when the built-in function for a data type has the valid type of arguments, but the arguments have invalid values specified.
RuntimeError	Raised when a generated error does not fall into any category.
NotImplementedError	Raised when an abstract method that needs to be implemented in an inherited class is not actually implemented.

HANDLING EXCEPTIONS

Python provides two very important features to handle any unexpected error in your Python programs and to add debugging capabilities in them.

Exception Handling:

- An exception is an event, which occurs during the execution of a program that disrupts the normal flow of the program's instructions. In general, when a Python script encounters a situation that it cannot cope with, it raises an exception. An exception is a Python object that represents an error.
- When a Python script raises an exception, it must either handle the exception immediately otherwise it terminates and quits.
- If you have some suspicious code that may raise an exception, you can defend your program by placing the suspicious code in a **try:** block. After the **try:** block, include an **except:** statement, followed by a block of code which handles the problem as elegantly as possible.

Python Exception clause

Try Except Finally clause

The **try** block lets you test a block of code for errors.

The **except** block lets you handle the error.

The **finally** block lets you execute code, regardless of the result of the **try-** and **except** blocks.

Exception Handling

When an error occurs or exception, Python will normally stop and generate an error message. These exceptions can be handled using the **try** statement:

Example

The **try** block will generate an exception, because **x** is not defined:

```
try:  
    print(x)  
except:  
    print("An exception occurred")
```

Output:

[REDACTED]

Since the try block raises an error, the except block will be executed.

Without the try block, the program will crash and raise an error:

Example

This statement will raise an error, because x is not defined:

```
print(x)
```

Output:

The output of the program is redacted with a black box.

Many Exceptions

You can define as many exception blocks as you want, e.g. if you want to execute a special block of code for a special kind of error:

Example

Print one message if the try block raises a NameError and another for other errors:

```
try:  
    print(x)  
except NameError:  
    print("Variable x is not defined")  
except:  
    print("Something else went wrong")
```

Output:

The output of the program is redacted with a black box.

Else

You can use the `else` keyword to define a block of code to be executed if no errors were raised:

Example

In this example, the try block does not generate any error:

```
try:
    print("Hello")
except:
    print("Something went wrong")
else:
    print("Nothing went wrong")
```

Output:

```
Hello
Something went wrong
```

Finally

The `finally` block, if specified, will be executed regardless if the `try` block raises an error or not.

Example

```
try:
    print(x)
except:
    print("Something went wrong")
finally:
    print("The 'try except' is finished")
```

Output:

```
Traceback (most recent call last):
  File "main.py", line 2, in <module>
    print(x)
NameError: name 'x' is not defined
```

This can be useful to close objects and clean up resources:

Example

Try to open and write to a file that is not writable:

```
try:
    f = open("demofile.txt")
    f.write("Lorum Ipsum")
except:
    print("Something went wrong when writing to the file")
finally:
    f.close()
```

Output:

```
Traceback (most recent call last):
  File "main.py", line 2, in <module>
    f = open("demofile.txt")
  File "main.py", line 3, in <module>
    f.write("Lorum Ipsum")
PermissionError: [Errno 13] Permission denied: 'demofile.txt'
```

The program can continue, without leaving the file object open.

Raise an exception

As a Python developer you can choose to throw an exception if a condition occurs.

To throw (or raise) an exception, use the `raise` keyword.

Example

Raise an error and stop the program if x is lower than 0:

```
x = -1

if x < 0:
    raise Exception("Sorry, no numbers below zero")
```

Output:

```
Traceback (most recent call last):
  File "demo_ref_keyword_raise2.py", line 4, in <module>
    raise Exception("Sorry, no numbers below zero")
Exception: Sorry, no numbers below zero
```

The `raise` keyword is used to raise an exception.

You can define what kind of error to raise, and the text to print to the user.

Example

Raise a `TypeError` if x is not an integer:

```
x = "hello"

if not type(x) is int:
    raise TypeError("Only integers are allowed")
```

Output:

```
Traceback (most recent call last):
  File "demo_ref_keyword_raise2.py", line 4, in <module>
    raise TypeError("Only integers are allowed")
TypeError: Only integers are allowed
```

If you need to determine whether an exception was raised but don't intend to handle it, a simpler form of the `raise` statement allows you to re-raise the exception:

```
try:
    raise NameError('HiThere')
except NameError:
    print('An exception flew by!')
    raise
```

Output:

```
An exception flew by!
Traceback (most recent call last):
  File "C:\Users\narmadha\Desktop\INDU\Python Programming\Python
example\hello.py", line 2, in <module>
    raise NameError('HiThere')
NameError: HiThere
```

User defined Exception:

- Python has many built-in exceptions which forces your program to output an error when something in it goes wrong. However, sometimes you may need to create custom exceptions that serves your purpose.
- In Python, users can define such exceptions by creating a new class. This exception class has to be derived, either directly or indirectly, from `Exception` class. Most of the built-in exceptions are also derived from this class.

```
>>> class CustomError(Exception):
...     pass
...
>>> raise CustomError
Traceback (most recent call last):
...
__main__.CustomError

>>> raise CustomError("An error occurred")
Traceback (most recent call last):
...
__main__.CustomError: An error occurred
```

- Here, we have created a user-defined exception called `CustomError` which is derived from the `Exception` class. This new exception can be raised, like other exceptions, using the `raise` statement with an optional error message.
- User-defined exception class can implement everything a normal class can do, but we generally make them simple and concise. Most implementations declare a custom base class and derive others exception classes from this base class. This concept is made clearer in the following example.

Example: User-Defined Exception in Python

In this example, we will illustrate how user-defined exceptions can be used in a program to raise and catch errors. This program will ask the user to enter a number until they guess a stored number correctly. To help them figure it out, hint is provided whether their guess is greater than or less than the stored number.

```
# define Python user-defined exceptions
class Error(Exception):
    """Base class for other exceptions"""
    pass

class ValueTooSmallError(Error):
    """Raised when the input value is too small"""
    pass

class ValueTooLargeError(Error):
    """Raised when the input value is too large"""
    pass

# our main program
# user guesses a number until he/she gets it right

# you need to guess this number
number = 10

while True:
    try:
        i_num = int(input("Enter a number: "))
        if i_num < number:
            raise ValueTooSmallError
        elif i_num > number:
```

```
        raise ValueError
    break
except ValueError:
    print("This value is too small, try again!")
    print()
except ValueError:
    print("This value is too large, try again!")
    print()

print("Congratulations! You guessed it correctly.")
```

Here is a sample run of this program. **OUTPUT**

```
Enter a number: 12
This value is too large, try again!

Enter a number: 0
This value is too small, try again!

Enter a number: 8
This value is too small, try again

Enter a number: 10
Congratulations! You guessed it correctly.
```

Here, we have defined a base class called **Error**.

The other two exceptions (**ValueTooSmallError** and **ValueTooLargeError**) that are actually raised by our program are derived from this class. This is the standard way to define user-defined exceptions in Python programming.

III. TURTLE GRAPHICS USING PYTHON

Turtle is a Python library to draw graphics. After we import Turtle we can give commands like forward, backward, right, left etc. These commands will draw different shapes.

Simple Turtle Commands

- ❖ forward(10) → It moves the turtle (arrow) forward by 10 pixels.
- ❖ backward(5) → It moves the turtle (arrow) backward by 5 pixels
- ❖ right(35) → It moves the turtle (arrow) clockwise by an angle of 35 degrees.
- ❖ left(55) → It moves the turtle (arrow) counter-clockwise by an angle of 55 degrees
- ❖ goto(x,y) → It moves the turtle (arrow) to the position x, y
- ❖ dot() → It creates a dot in the current position.
- ❖ shape('circle') → It draws a circle shape.

Examples

Let's see some drawings using some simple commands.

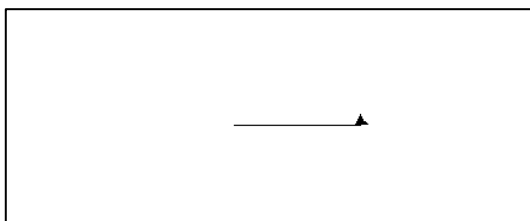
Draw a Star

In the below program we draw a star. We choose appropriate steps to move the cursor forward and then right continuously to get this result.

Example

```
import turtle
star = turtle.Turtle()
for i in range(100):
    star.forward(100)
    star.right(144)
turtle.done()
```

Output



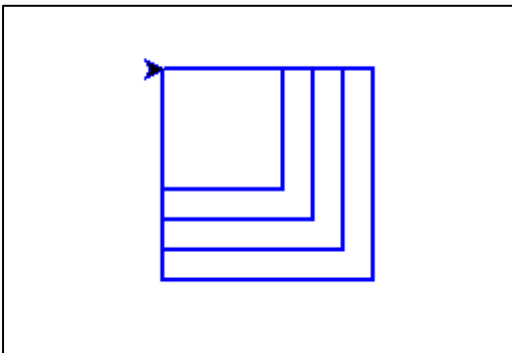
Multiple Squares

In the next example we see the drawing of multiple squares all starting from a common point. We use the usual simple commands to go forward, backward and then turn 90 degrees.

Example

```
import turtle
mult_square=turtle.Turtle()
def Multiple_Squares(length, colour):
    mult_square.pencolor(colour)
    mult_square.pensize(2)
    mult_square.forward(length)
    mult_square.right(90)
    mult_square.forward(length)
    mult_square.right(90)
    mult_square.forward(length)
    mult_square.right(90)
    mult_square.forward(length)
    mult_square.right(90)
    mult_square.setheading(360)
    for i in range(60,120,15):
        Multiple_Squares(i,'blue')
    turtle.done
```

Running the above code gives us the following result

Output

IV. IMAGE PROCESSING

Image processing allows us to transform and manipulate thousands of images at a time and extract useful insights from them. It has a wide range of applications in almost every field. Python is one of the widely used programming languages for this purpose.

what is an image?

- ✓ An image can be represented as a 2D function $F(x,y)$ where x and y are spatial coordinates.
- ✓ The amplitude of F at a particular value of x,y is known as the intensity of an image at that point.
- ✓ If x,y , and the amplitude value is finite then we call it a digital image.
- ✓ It is an array of pixels arranged in columns and rows. Pixels are the elements of an image that contain information about intensity and color.
- ✓ An image can also be represented in 3D where x,y , and z become spatial coordinates.
- ✓ Pixels are arranged in the form of a matrix.

Types of image processing

1. Morphological Image Processing

- ❖ Morphological image processing tries to remove the imperfections from the binary images because binary regions produced by simple thresholding can be distorted by noise. It also helps in smoothing the image using opening and closing operations.
- ❖ Morphological operations can be extended to grayscale images.
- ❖ It consists of non-linear operations related to the structure of features of an image. It depends on the related ordering of pixels but on their numerical values.
- ❖ This technique analyses an image using a small template known as a structuring element which is placed in different possible locations in the image and is compared with the corresponding neighborhood pixels. A structuring element is a small matrix with 0 and 1 values.

2. Gaussian Image Processing

- ❖ Gaussian blur which is also known as gaussian smoothing, is the result of blurring an image by a Gaussian function.
- ❖ It is used to reduce image noise and reduce details. The visual effect of this blurring technique is similar to looking at an image through the translucent screen. It is sometimes used in computer vision for image enhancement at different scales or as a data augmentation technique in deep learning.

3. Fourier Transform In Image Processing

Fourier transform breaks down an image into sine and cosine components. It has multiple applications like image reconstruction, image compression, or image filtering. Since we are talking about images, we will take discrete Fourier transform into consideration.

Let's consider a sinusoid, it comprises three things:

- Magnitude – related to contrast
- Spatial frequency – related to brightness
- Phase – related to color information

4. Edge Detection In Image Processing

- ❖ Edge detection is an image processing technique for finding the boundaries of objects within images. It works by detecting discontinuities in brightness.
- ❖ This could be very beneficial in extracting useful information from the image because most of the shape information is enclosed in the edges. Classic edge detection methods work by detecting discontinuities in the brightness.
- ❖ It can rapidly react if some noise is detected in the image while detecting the variations of grey levels. Edges are defined as the local maxima of the gradient.

5. Wavelet Image Processing

- ❖ Wavelets take both time and frequency into consideration. This transform is apt for non-stationary signals.
- ❖ We know that edges are one of the important parts of the image, while applying the traditional filters it's been noticed that noise gets removed but the image gets blurry.
- ❖ The wavelet transform is designed in such a way that we get good frequency resolution for low-frequency components.

Image processing using cv2

(a) Rotate an Image

First of all, import the cv2 module.

```
import cv2
```

Now to read the image, use the imread() method of the cv2 module, specify the path to the image in the arguments and store the image in a variable as below.

```
img = cv2.imread("pyimg.jpg")
```

The image is now treated as a matrix with rows and columns values stored in img.

Actually, if you check the type of the img, it will give you the following result:

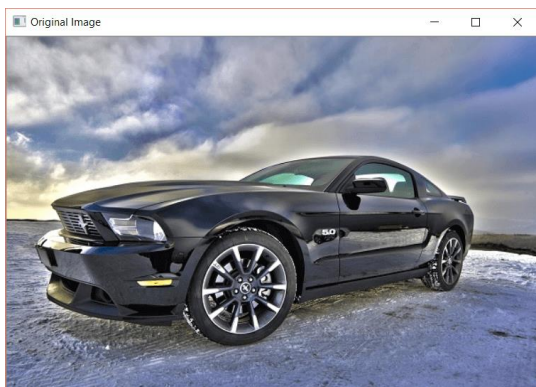
```
>>>print(type(img))  
  
<class 'numpy.ndarray'>
```

It's a NumPy array! That's why image processing using OpenCV is so easy. All the time you are working with a NumPy array.

To display the image, you can use the `imshow()` method of `cv2`.

```
cv2.imshow('Original Image', img)  
  
cv2.waitKey(0)
```

The `waitkey` functions take time as an argument in milliseconds as a delay for the window to close. Here we set the time to zero to show the window forever until we close it manually.



To rotate this image, you need the width and the height of the image because you will use them in the rotation process as you will see later.

```
height, width = img.shape[0:2]
```

The `shape` attribute returns the height and width of the image matrix. If you print

```
img.shape[0:2]
```

you will have the following output:

```
>>> import cv2  
>>> img = cv2.imread("pyimg.jpg")  
>>> print(img.shape[0:2])  
(432, 768)
```

Okay, now we have our image matrix and we want to get the rotation matrix. To get the rotation matrix, we use the *getRotationMatrix2D()* method of cv2. The syntax of *getRotationMatrix2D()* is:

```
cv2.getRotationMatrix2D(center, angle, scale)
```

Here the **center** is the center point of rotation, the **angle** is the angle in degrees and **scale** is the scale property which makes the image fit on the screen.

To get the rotation matrix of our image, the code will be:

```
rotationMatrix = cv2.getRotationMatrix2D((width/2, height/2), 90, .5)
```

The next step is to rotate our image with the help of the rotation matrix.

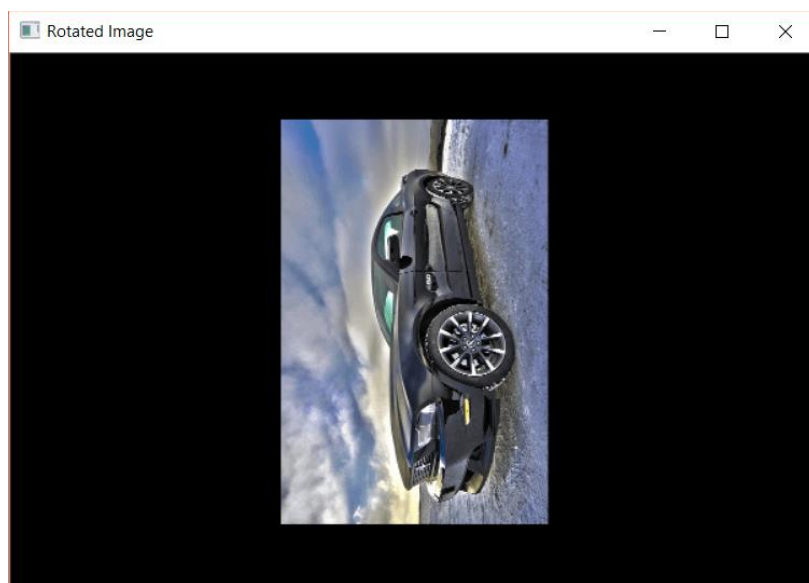
To rotate the image, we have a cv2 method named *warpAffine* which takes the original image, the rotation matrix of the image and the width and height of the image as arguments.

```
rotatedImage = cv2.warpAffine(img, rotationMatrix, (width, height))
```

The rotated image is stored in the rotatedImage matrix. To show the image, use *imshow()* as below:

```
cv2.imshow('Rotated Image', rotatedImage)  
cv2.waitKey(0)
```

After running the above lines of code, you will have the following output:



(b) Crop an Image

First, we need to import the cv2 module and read the image and extract the width and height of the image:

```
import cv2

img = cv2.imread("pyimg.jpg")

height, width = img.shape[0:2]
```

Now get the starting and ending index of the row and column. This will define the size of the newly created image. For example, start from row number 10 till row number 15 will give the height of the image.

Similarly, start from column number 10 until column number 15 will give the width of the image.

You can get the starting point by specifying the percentage value of the total height and the total width. Similarly, to get the ending point of the cropped image, specify the percentage values as below:

```
startRow = int(height*.15)

startCol = int(width*.15)

endRow = int(height*.85)

endCol = int(width*.85)
```

Now map these values to the original image. Note that you have to cast the starting and ending values to integers because when mapping, the indexes are always integers.

```
croppedImage = img[startRow:endRow, startCol:endCol]
```

Here we specified the range from starting to ending of rows and columns.

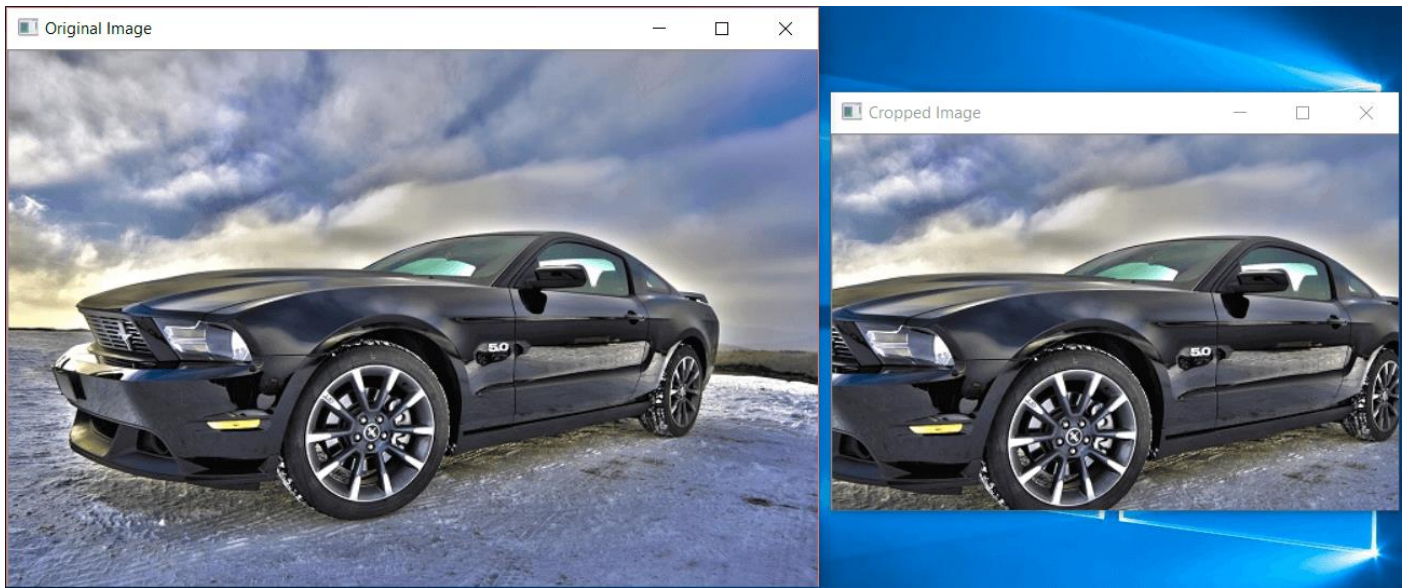
Now display the original and cropped image in the output:

```
cv2.imshow('Original Image', img)

cv2.imshow('Cropped Image', croppedImage)

cv2.waitKey(0)
```

The result will be as follows:



(c) Resize an Image

To resize an image, you can use the `resize()` method of openCV. In the `resize` method, you can either specify the values of x and y axis or the number of rows and columns which tells the size of the image.

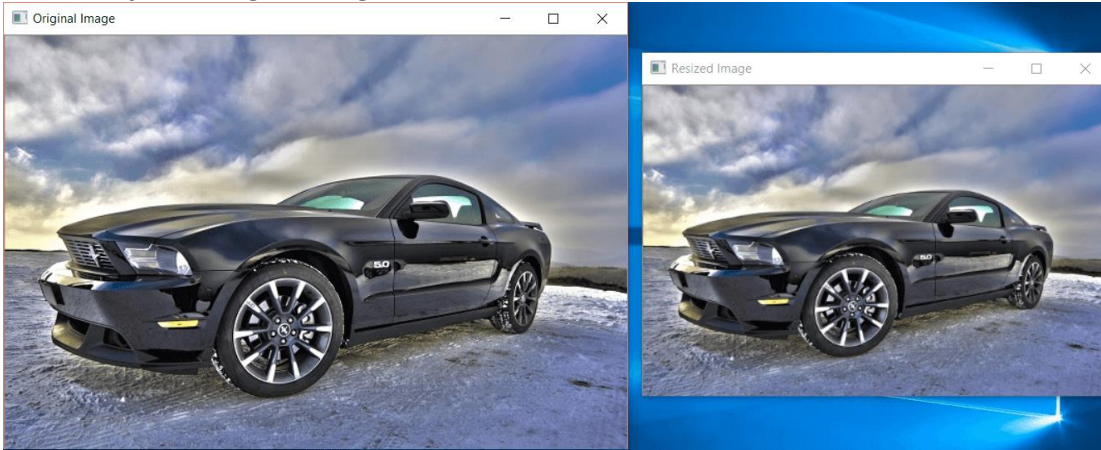
Import and read the image:

```
import cv2  
  
img = cv2.imread("pyimg.jpg")
```

Now using the `resize` method with axis values:

```
newImg = cv2.resize(img, (0,0), fx=0.75, fy=0.75)  
  
cv2.imshow('Resized Image', newImg)  
  
cv2.waitKey(0)
```

The result will be as follows:

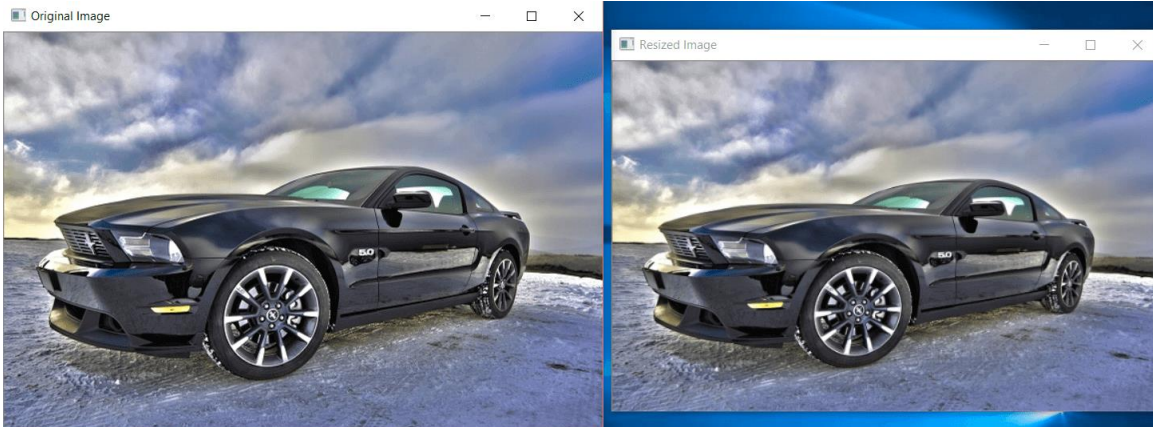


Now using the row and column values to resize the image:

```
newImg = cv2.resize(img, (550, 350))  
cv2.imshow('Resized Image', newImg)  
cv2.waitKey(0)
```

We say we want 550 columns (the width) and 350 rows (the height).

The result will be:



(d) Adjust Image Contrast

In Python OpenCV module, there is no particular function to adjust image contrast but the official documentation of OpenCV suggests an equation that can perform image brightness and image contrast both at the same time.

```
new_img = a * original_img + b
```

Here a is alpha which defines the contrast of the image. If a is greater than 1, there will be higher contrast.

If the value of a is between 0 and 1 (smaller than 1 but greater than 0), there would be lower contrast. If a is 1, there will be no contrast effect on the image.

b stands for beta. The values of b vary from -127 to +127.

To implement this equation in Python OpenCV, you can use the `addWeighted()` method. We use `addWeighted()` method as it generates the output in the range of 0 and 255 for a 24-bit color image.

The syntax of `addWeighted()` method is as follows:

```
cv2.addWeighted(source_img1, alpha1, source_img2, alpha2, beta)
```

This syntax will blend two images, the first source image (`source_img1`) with a weight of `alpha1` and second source image (`source_img2`).

If you only want to apply contrast in one image, you can add a second image source as zeros using NumPy.

Let's work on a simple example. Import the following modules:

```
import cv2  
import numpy as np
```

Read the original image:

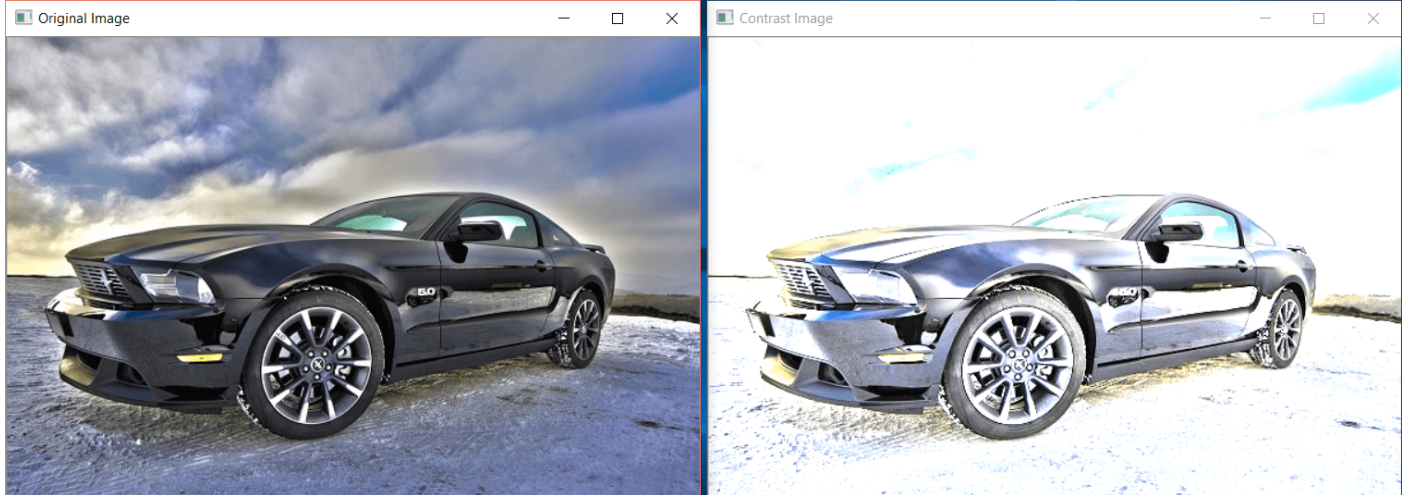
```
img = cv2.imread("pyimg.jpg")
```

Now apply the contrast. Since there is no other image, we will use the `np.zeros` which will create an array of the same shape and data type as the original image but the array will be filled with zeros.

```
contrast_img = cv2.addWeighted(img, 2.5, np.zeros(img.shape, img.dtype), 0, 0)  
cv2.imshow('Original Image', img)  
cv2.imshow('Contrast Image', contrast_img)  
cv2.waitKey(0)
```

In the above code, the brightness is set to 0 as we only want to apply contrast.

The comparison of the original and contrast image is as follows:



(e) **Make an image blurry**

Gaussian Blur To make an image blurry, you can use the `GaussianBlur()` method of OpenCV. The `GaussianBlur()` uses the Gaussian kernel. The height and width of the kernel should be a positive and an odd number. Then you have to specify the X and Y direction that is `sigmaX` and `sigmaY` respectively. If only one is specified, both are considered the same.

Consider the following example:

```
import cv2

img = cv2.imread("pyimg.jpg")

blur_image = cv2.GaussianBlur(img, (7,7), 0)

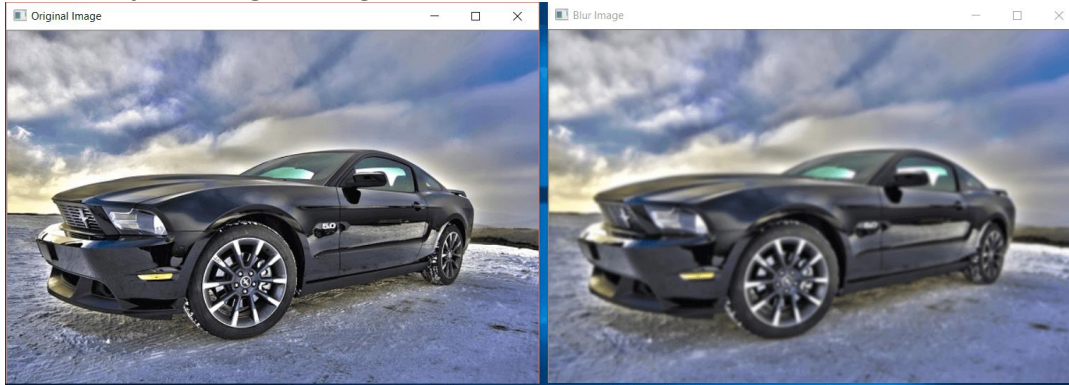
cv2.imshow('Original Image', img)

cv2.imshow('Blur Image', blur_image)

cv2.waitKey(0)
```

In the above snippet, the actual image is passed to `GaussianBlur()` along with height and width of the kernel and the X and Y directions.

The comparison of the original and blurry image is as follows:



(g) Detect Edges

To detect the edges in an image, you can use the Canny() method of cv2 which implements the Canny edge detector. The Canny edge detector is also known as the *optimal detector*.

The syntax to Canny() is as follows:

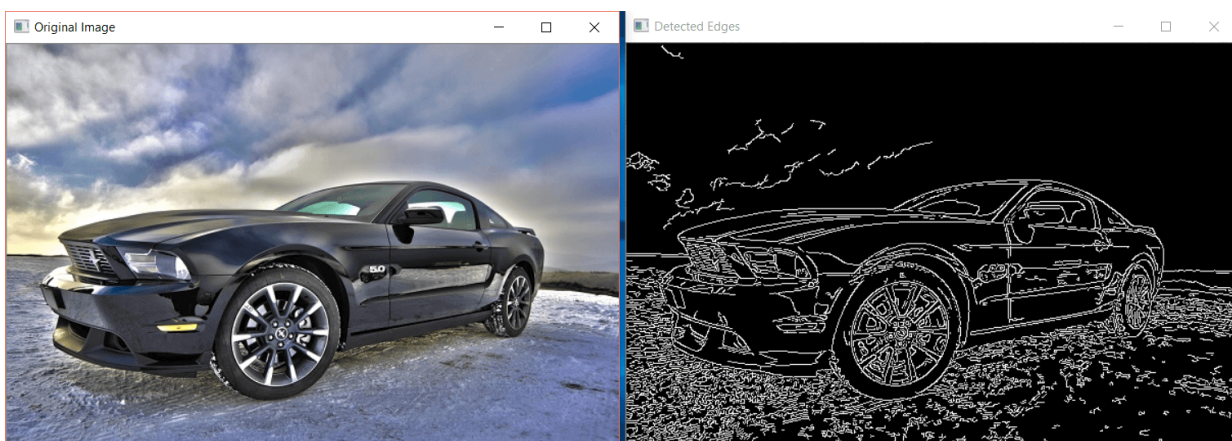
```
cv2.Canny(image, minVal, maxVal)
```

Here minVal and maxVal are the minimum and maximum intensity gradient values respectively.

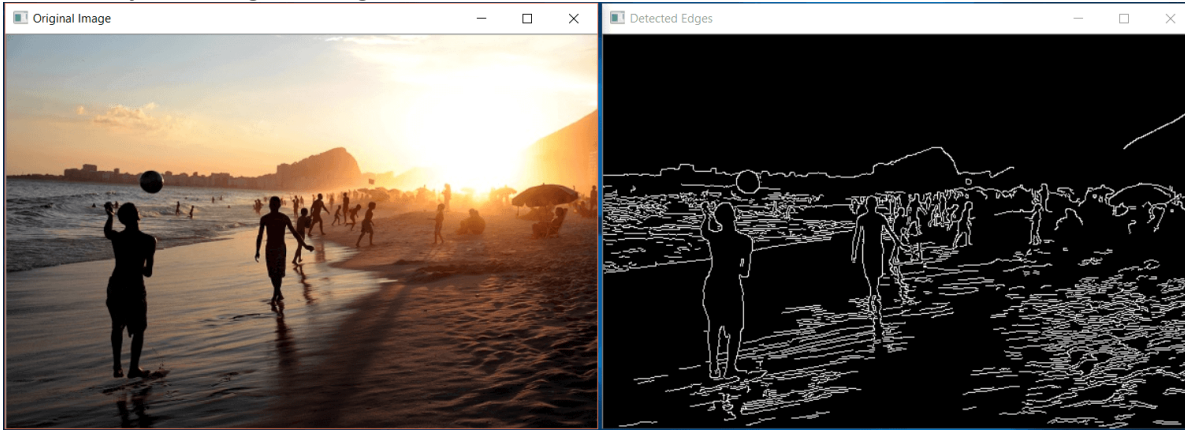
Consider the following code:

```
import cv2
img = cv2.imread("pyimg.jpg")
edge_img = cv2.Canny(img,100,200)
cv2.imshow("Detected Edges", edge_img)cv2.waitKey(0)
```

The output will be the following:



Here is the result of the above code on another image:



2D-GRAPHICS-COLORS-SHAPES

turtle is an inbuilt module in python. It provides drawing using a screen (cardboard) and turtle (pen). To draw something on the screen, we need to move the turtle. To move turtle, there are some functions i.e forward(), backward(), etc.

To fill the colors in the shapes drawn by turtle, turtle provides three functions –

fillcolor(): This helps to choose the color for filling the shape. It takes the input parameter as the color name or hex value of the color and fills the upcoming closed geographical objects with the chosen color. Color names are basic color names i.e. red, blue, green, orange.

The hex value of color is a string(starting with '#') of hexadecimal numbers i.e. #RRGGBB. R, G, and B are the hexadecimal numbers (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F).

begin_fill(): This function tells turtle that all upcoming closed graphical objects needed to be filled by the chosen color.

end_fill(): this function tells turtle to stop the filling upcoming closed graphical objects.

Drawing Color Filled Square:

```
# draw color-filled square in turtle

import turtle

# creating turtle pen
t = turtle.Turtle()

# taking input for the side of the square
s = int(input("Enter the length of the side of the square: "))

# taking the input for the color
col = input("Enter the color name or hex value of color(# RRGGBB): ")

# set the fillcolor
t.fillcolor(col)

# start the filling color
t.begin_fill()

# drawing the square of side s
for _ in range(4):
    t.forward(s)
    t.right(90)
```

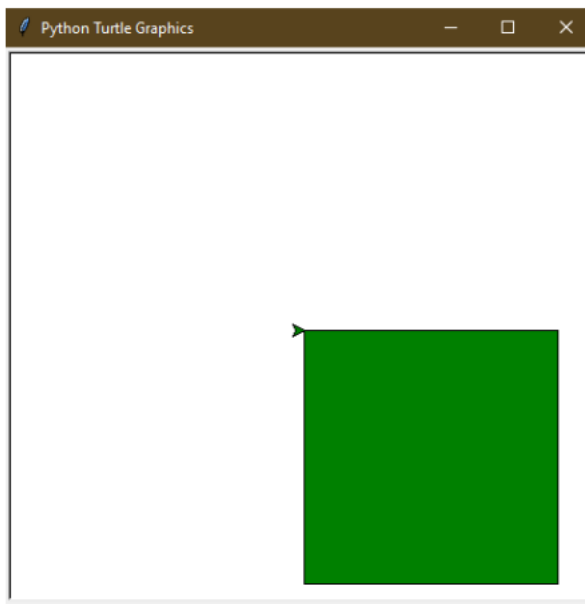
```
# ending the filling of the color  
t.end_fill()
```

Input :

200

green

Output :



Drawing Color Filled Triangle:

```
# draw color filled triangle in turtle  
  
import turtle  
  
# creating turtle pen  
t = turtle.Turtle()  
  
# taking input for the side of the triangle  
s = int(input("Enter the length of the side of the triangle: "))  
  
# taking the input for the color  
col = input("Enter the color name or hex value of color(#  
RRGGBB): ")  
  
# set the fillcolor  
t.fillcolor(col)  
  
# start the filling color  
t.begin_fill()
```

```
# drawing the triangle of side s
for _ in range(3):
    t.forward(s)
    t.right(-120)

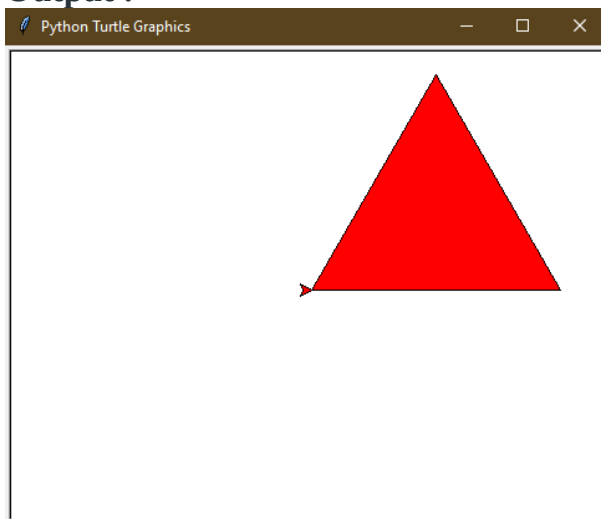
# ending the filling of the color
t.end_fill()
```

Input :

200

red

Output :



Drawing Color Filled Hexagon:

```
# draw color-filled hexagon in turtle

import turtle

# creating turtle pen
t = turtle.Turtle()

# taking input for the side of the hexagon
s = int(input("Enter the length of the side of the hexagon: "))

# taking the input for the color
col = input("Enter the color name or hex value of color(#RRGGBB): ")

# set the fillcolor
t.fillcolor(col)

# start the filling color
t.begin_fill()

# drawing the hexagon of side s
```

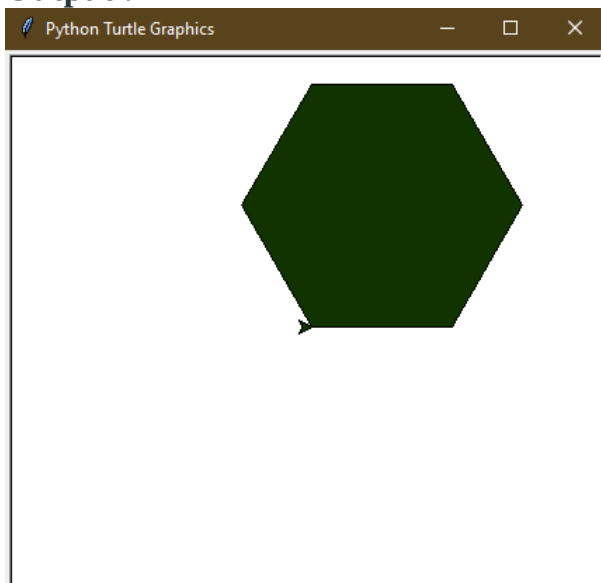
```
for _ in range(6):  
    t.forward(s)  
    t.right(-60)  
  
# ending the filling of the color  
t.end_fill()
```

Input :

100

#113300

Output :



Drawing Color Filled Star:

```
# draw color filled star in turtle  
  
import turtle  
  
# creating turtle pen  
t = turtle.Turtle()  
  
# taking input for the side of the star  
s = int(input("Enter the length of the side of the star: "))  
  
# taking the input for the color  
col = input("Enter the color name or hex value of color(#  
RRGGBB): ")  
  
# set the fillcolor  
t.fillcolor(col)  
  
# start the filling color  
t.begin_fill()
```

```
# drawing the star of side s
for _ in range(5):
    t.forward(s)
    t.right(144)

# ending the filling of color
t.end_fill()
```

Input :

200

#551122

Output :

